

Dropout Regularization

Theory, Implementation, and Applications

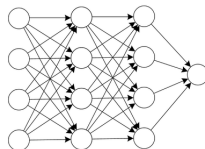
Moin Mostakim

What is Dropout?

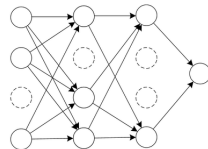
Key Idea

Randomly "drop" (set to zero) neurons during training to prevent co-adaptation

- Introduced by Srivastava et al. (2014)
- Simple yet effective regularization
- Works for feedforward and recurrent networks
- Each training iteration: different "thinned" network



(a) Standard Neural Network



(b) Network after Dropout

Mathematical Formulation

During Training

For layer l with activation $a^{(l)}$:

$$\tilde{a}^{(l)} = a^{(l)} \odot m^{(l)}$$

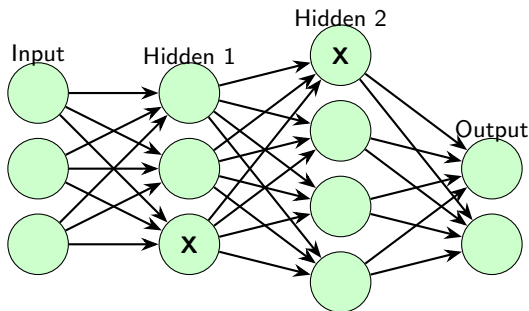
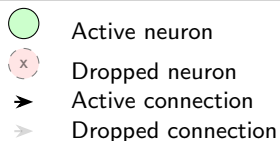
where $m^{(l)} \sim \text{Bernoulli}(p)$ is the **mask vector**:

$$m_i^{(l)} = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

Hyperparameters

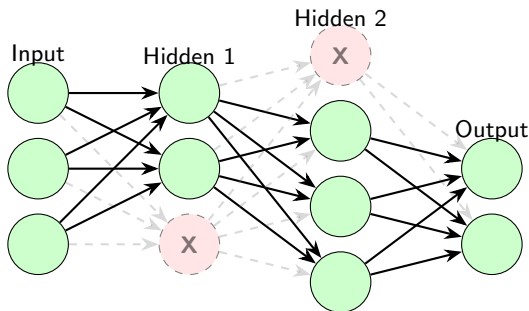
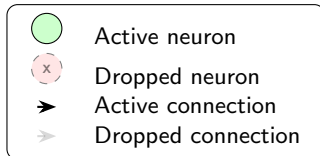
- p : Dropout keep probability (typically 0.5-0.8)
- $1 - p$: Dropout rate

Dropout Visualization with Animation



Before Dropout: All neurons and connections active

Dropout Visualization with Animation



After Dropout: Neurons marked with X are dropped

Inference with Dropout

During Testing/Inference

- No dropout applied
- All neurons active
- But: Need to account for missing activations during training

Approximate Inference Methods

- 1 **Standard Scaling:** Multiply weights by p
- 2 **Inverted Dropout:** Scale activations during training by $1/p$
- 3 **Monte Carlo Dropout:** Sample multiple forward passes

Inverted Dropout (Recommended)

Training Phase

$$\tilde{a}^{(l)} = \frac{a^{(l)} \odot m^{(l)}}{p}$$

Testing Phase

$$\tilde{a}^{(l)} = a^{(l)}$$

Advantages

- Same forward pass at test time
- No weight scaling needed
- More stable during training

Mathematical Justification: Ensemble Interpretation

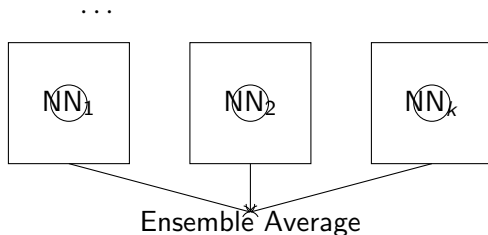
Key Insight

Dropout trains an **ensemble** of 2^n thinned networks

For network with n units:

Number of subnetworks = 2^n

- Each subnetwork sees different data
- Final prediction: average over ensemble
- Reduces variance without increasing bias



Expected Output Derivation

Goal: Ensure $\mathbb{E}[\tilde{a}] = a$

Let a be pre-dropout activation, $m \sim \text{Bernoulli}(p)$

Without Scaling

$$\mathbb{E}[\tilde{a}] = \mathbb{E}[a \odot m] = a \cdot \mathbb{E}[m] = a \cdot p$$

Expected value is scaled by p !

With Scaling (Inverted Dropout)

$$\tilde{a} = \frac{a \odot m}{p}$$

$$\mathbb{E}[\tilde{a}] = \mathbb{E}\left[\frac{a \odot m}{p}\right] = \frac{a}{p} \cdot \mathbb{E}[m] = \frac{a}{p} \cdot p = a$$

Perfect preservation of expected value!

Gradient Flow with Dropout

Forward Pass

$$z^{(l+1)} = W^{(l+1)} \tilde{a}^{(l)} + b^{(l+1)}$$

$$\tilde{a}^{(l)} = \frac{a^{(l)} \odot m^{(l)}}{p}$$

Backward Pass

$$\frac{\partial L}{\partial a^{(l)}} = \frac{\partial L}{\partial \tilde{a}^{(l)}} \odot \frac{m^{(l)}}{p}$$

Gradients only flow through active neurons!

Important

Only apply dropout to **hidden layers**, not input or output layers!

Dropout Variants

Standard Dropout

- Random neuron dropout
- Same p for all layers
- Most common variant

Spatial Dropout

- For convolutional networks
- Drop entire feature maps
- Better for spatial correlations

DropConnect

- Drop weights instead of activations
- More aggressive regularization
- W_{ij} set to zero with probability p

Alpha Dropout

- For self-normalizing networks (SELU)
- Maintains mean and variance
- Preserves self-normalizing property

Implementation in PyTorch

Basic Dropout Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)
        self.dropout = nn.Dropout(p=0.5)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.dropout(x) # Apply dropout
        x = F.relu(self.fc2(x))
        x = self.dropout(x) # Apply dropout
        x = self.fc3(x)
        return x
```

Implementation Details

Training vs Evaluation Mode

```
model = Net()

# Training mode: dropout active
model.train()
output = model(input_data)
loss = criterion(output, target)
loss.backward()

# Evaluation mode: dropout inactive
model.eval()
with torch.no_grad():
    predictions = model(test_data)
```

Custom Dropout Rate per Layer

```
class CustomNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.dropout1 = nn.Dropout(0.2) # Input layer
        self.dropout2 = nn.Dropout(0.5) # Hidden layer 1
        self.dropout3 = nn.Dropout(0.5) # Hidden layer 2
```

Mathematical Analysis: Variance

Effect on Activation Variance

Let $a_i \sim \mathcal{N}(\mu, \sigma^2)$ i.i.d., $m_i \sim \text{Bernoulli}(p)$

With inverted dropout:

$$\tilde{a}_i = \frac{a_i m_i}{p}$$

$$\mathbb{E}[\tilde{a}_i] = \mu$$

$$\text{Var}(\tilde{a}_i) = \frac{\sigma^2}{p} + \mu^2 \frac{1-p}{p}$$

Implication

- Variance increases when $p < 1$
- Acts as noise injection
- Equivalent to data augmentation

Dropout as Adaptive L2 Regularization

Connection to L2 Regularization

For linear regression with dropout:

$$\mathbb{E}_m[J(\theta)] = J(\theta) + \frac{1-p}{2p} \sum_{i=1}^n \theta_i^2$$

where $\mathbb{E}_m$ is expectation over dropout masks.

Interpretation

- Dropout $\approx$ Adaptive L2 regularization
- Penalty proportional to $(1-p)/p$
- Larger dropout rate $\Rightarrow$ stronger regularization

Note

This equivalence is approximate and holds best for linear models

Choosing Dropout Rates

Layer Type	Typical p	Rationale
Input Layer	0.8-1.0	Preserve input information
Hidden Layers	0.5-0.8	Balance regularization
Wide Layers	0.2-0.5	More regularization needed
Narrow Layers	0.7-0.9	Less regularization needed
Output Layer	1.0	Never drop output neurons

Table: Recommended dropout rates

Guidelines

- Start with $p = 0.5$ for hidden layers
- Increase p if underfitting
- Decrease p if overfitting
- Use validation set to tune

Dropout in Convolutional Networks

Standard Dropout Issues

- Breaks spatial correlations
- Less effective for CNNs
- Can harm performance

Spatial Dropout Solution

- Drop entire feature maps
- Implemented as Dropout2D in PyTorch
- Preserves spatial structure

```
# PyTorch implementation  
self.dropout = nn.Dropout2d(p=0.5) # Spatial dropout
```

Practical Tips and Best Practices

Do:

- Use inverted dropout
- Apply after activation
- Tune p per layer
- Combine with batch norm
- Use for fully connected layers

Don't:

- Apply to input/output layers
- Use with small datasets
- Combine with strong L2
- Use in final layers
- Forget `model.train()/eval()`

Combination with Other Techniques

- **BatchNorm + Dropout:** Can be tricky, order matters
- **Dropout + L2:** Often redundant
- **Dropout + Data Augmentation:** Synergistic effect

Experimental Results

Key Findings from Original Paper

- Reduces overfitting significantly
- Improves generalization
- Works across architectures
- Simple to implement

Dataset	No Dropout	With Dropout	Improvement
MNIST	1.60%	1.25%	0.35%
CIFAR-10	18.50%	15.60%	2.90%
ImageNet	37.50%	33.80%	3.70%

Table: Error rates with and without dropout

Bayesian Interpretation

Dropout can be viewed as approximate Bayesian inference:

$$p(y|x, D) \approx \frac{1}{T} \sum_{t=1}^T p(y|x, \theta_t)$$

where θ_t are sampled dropout masks.

Uncertainty Estimation

```
# Monte Carlo dropout predictions
model.train() # Keep dropout active!
predictions = []
for _ in range(num_samples):
    pred = model(input_data)
    predictions.append(pred)

mean_pred = torch.stack(predictions).mean(0)
uncertainty = torch.stack(predictions).std(0)
```

Summary

Key Points

- 1 Dropout prevents neuron co-adaptation
- 2 Equivalent to training ensemble of networks
- 3 Use inverted dropout for stability
- 4 Scale activations by $1/p$ during training
- 5 No scaling needed during inference

When to Use Dropout

- Large neural networks
- Limited training data
- Signs of overfitting
- Fully connected layers

When to Avoid

- Very small networks
- Already using strong regularization
- Convolutional layers (use Spatial Dropout instead)